

Documento técnico de entrega — Blacklist API

1. Objetivo del documento

Este documento recoge la explicación técnica del proyecto y la evidencia disponible para la entrega, teniendo en cuenta el error encontrado durante la validación, los requisitos del documento de entregables y la necesidad de dejar un trabajo centrado en:

- A. Configuración de base de datos.
- C. Cobertura de tests con al menos 90%.

Se aclara que no hay imágenes dentro del repositorio disponible; por tanto, la evidencia se basa en el código del proyecto, la documentación técnica y la salida real de ejecución del entorno.

2. Resumen ejecutivo del proyecto

El proyecto es un microservicio Flask para gestionar una lista negra de correos electrónicos (Blacklist API). Tiene como objetivo centralizar la validación y persistencia de direcciones de correo bloqueadas para prevenir envíos no autorizados.

Dentro del alcance del proyecto se contemplan:

- API REST con Python y Flask.
- Persistencia con PostgreSQL.
- Validación de entradas y seguridad con JWT.
- Endpoints para crear y consultar registros de emails bloqueados.
- Cobertura de pruebas unitarias e integradas.
- Despliegue en nube pública.
- Documentación de configuración y despliegue.

La documentación base del entregable se encuentra en Requerimientos_Blacklist_API.md, la cual define explícitamente que el proyecto debe cumplir requisitos tanto funcionales como no funcionales, además de evidencia de cobertura y despliegue.

3. Requisitos de entregables que exige la especificación

La especificación de entregables exige lo siguiente:

1. Código fuente del proyecto con la estructura del microservicio en Flask.
2. Evidencia ejecutada de pruebas automatizadas con cobertura mínima del 90%.

3. Guía de despliegue paso a paso con capturas de pantalla.
4. Configuración de base de datos en la nube.
5. Despliegue del proyecto en la nube.
6. Verificación funcional del servicio con llamadas HTTP y validación final.

Además, el documento de requisitos indica que la evaluación de la entrega se estructura así:

- Stack tecnológico: 20%
- Endpoints y lógica: 30%
- Cobertura de pruebas: 20%
- Despliegue en nube: 20%
- Documentación: 10%

Esto confirma que el trabajo no debe centrarse únicamente en la nube, sino también en la base de datos y en la calidad del código validada con pruebas.

4. Error real detectado en la validación

Durante la ejecución de la prueba de cobertura se obtuvo este error:

```
cd /workspaces/blacklist_app && pytest --cov=src --cov-report=term-missing -q
```

Resultado real:

```
ImportError while loading conftest '/workspaces/blacklist_app/tests/conftest.py'
.
tests/conftest.py:8: in
  from src.db.database import db
src/db/database.py:2: in
  from flask_sqlalchemy import SQLAlchemy
E ModuleNotFoundError: No module named 'flask_sqlalchemy'
```

4.1. Qué significa el error

La aplicación no puede inicializar la capa de base de datos porque la dependencia `Flask-SQLAlchemy` no está instalada en el entorno actual. El archivo src/db/database.py importa directamente:

```
from flask_sqlalchemy import SQLAlchemy
```

Y además, el proyecto define la dependencia en pyproject.toml:

```
flask-sqlalchemy = "^3.1.1"
```

El problema, por tanto, no es del código de negocio sino del entorno de ejecución y de instalación de dependencias. La solución correcta es instalar las dependencias del proyecto leyendo el entorno con Poetry o el gestor correspondiente antes de correr pruebas o levantar la app.

5. Evidencia técnica del proyecto

5.1. Dependencias declaradas

La dependencia de SQLAlchemy está declarada en pyproject.toml:

```
[tool.poetry.dependencies]
python = "^3.10"
flask = "^3.0.0"
flask-sqlalchemy = "^3.1.1"
flask-restful = "^0.3.10"
flask-marshmallow = "^1.2.0"
...
```

5.2. Inicialización de base de datos

La inicialización se hace en src/db/database.py:

```
import os
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()

def init_db(app):
    database_url = os.environ.get("DATABASE_URL")
    if not database_url:
        raise ValueError("DATABASE_URL environment variable must be set")
    app.config["SQLALCHEMY_DATABASE_URI"] = database_url
    app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False
    db.init_app(app)
    return db
```

Esto confirma que la base de datos es un componente central del proyecto y debe configurarse correctamente antes de ejecutar pruebas o despliegue.

5.3. Configuración de la app

La app principal se define en src/main.py y carga la base de datos al inicio:

```
from .db.database import init_db, create_tables  
  
...  
init_db(app)  
  
...  
with app.app_context():  
    create_tables(app)
```

5.4. Cobertura requerida

En pyproject.toml aparece la regla de cobertura mínima exigida:

```
[tool.coverage.report]  
show_missing = true  
fail_under = 90
```

Esto refleja exactamente el requisito del entregable: mínimo 90%.

6. Qué se pide en el punto 1 del entregable

El punto 1, según la especificación, exige que el documento final explique y evidencie:

6.1. Código fuente del proyecto

Debe incluir la estructura general del proyecto, la organización por capas:

- src/
 - db/
 - models/
 - routes/
 - services/
 - repositories/
 - utils/
- tests/

6.2. Pruebas y cobertura

Se debe mostrar la ejecución real de `pytest` con `coverage` y evidenciar una cobertura igual o superior al 90%.

6.3. Base de datos

Debe incluir la configuración de la base de datos en entorno de despliegue, especialmente PostgreSQL y el parámetro ``DATABASE_URL``.

6.4. Nube y despliegue

Aunque el usuario indica que el trabajo está centrado en el punto A y en C, la documentación también debe incluir el despliegue en nube como parte de la entrega formal. Sin embargo, la prioridad técnica del proyecto está en:

- configuración de la base de datos,
- conexión entre app y BD,
- pruebas automatizadas,
- cobertura.

7. Enfoque correcto para el documento final

El documento debería estructurarse en estos bloques:

Bloque 1: Introducción

- Qué es la aplicación.
- Qué problema resuelve.
- Qué tecnologías usa.

Bloque 2: Arquitectura y componentes

- Flask API.
- SQLAlchemy.
- PostgreSQL.
- JWT.
- Rutas y servicios.

Bloque 3: Configuración de base de datos

- Instalación de PostgreSQL.
- Creación de la base ``blacklist_db``.
- Variables de entorno.
- Ejemplo de ``DATABASE_URL``.
- Migrations o creación de tablas.

Bloque 4: Error encontrado y corrección

- Error real: `ModuleNotFoundError: No module named 'flask\_sqlalchemy'`.
- Causa: entorno sin instalar dependencias.
- Acción a tomar: `poetry install` y re-ejecutar tests.

Bloque 5: Pruebas automatizadas y cobertura

- Evidencia de ejecución con `pytest`.
- Cobertura mínima esperada del 90%.
- Comando recomendado:

`poetry install`

`poetry run pytest --cov=src --cov-report=term-missing -q`

Bloque 6: Despliegue en nube

- PostgreSQL en nube.
- Variables de entorno.
- Servicio de cómputo (Docker, EC2, App Runner, Cloud Run, etc.).
- Validación del health check y llamadas HTTP.

Bloque 7: Conclusión

- El proyecto cumple la lógica funcional y la estructura exigida.
- El punto crítico a resolver es la instalación de dependencias y la validación de la conexión a la base de datos.

8. Recomendación práctica para la entrega

Para una entrega sólida, se recomienda dejar esto documentado claramente:

1. El error se detectó en la ejecución real del proyecto y no es teórico.
2. La causa fue la dependencia faltante de `flask\_sqlalchemy`.
3. La configuración de la base de datos es esencial y debe estar descrita.
4. La cobertura del 90% debe aparecer como evidencia de pruebas ejecutadas.
5. El despliegue en nube debe mencionarse, pero sin perder el foco en la base de datos y los tests.

9. Conclusión

El trabajo solicitado debe responder a tres elementos clave: configuración de base de datos, pruebas con cobertura del 90% y despliegue en nube. El proyecto ya muestra la estructura correcta y la intención del

entregable, pero la validación real revela un problema de entorno que debe documentarse y corregirse antes de entregar una versión final.

En otras palabras, el documento debe no solo explicar la solución, sino también dejar evidencia de la ejecución, del error y de la corrección necesaria para cumplir con la entrega.

10. Evidencia resumida

- Proyecto base: Blacklist API en Flask.
- Requisito de cobertura: 90% mínimo.
- Error encontrado: `ModuleNotFoundError: No module named 'flask_sqlalchemy'`.
- Archivos clave: pyproject.toml, src/db/database.py, src/main.py, tests/conftest.py.
- Documento base de requisitos: Requerimientos_Blacklist_API.md.